Exotic Agentic Development Strategies:
Everything old is new again

Stephen Shary



MOMENTUM
Developer Conference
powered by The Circuit

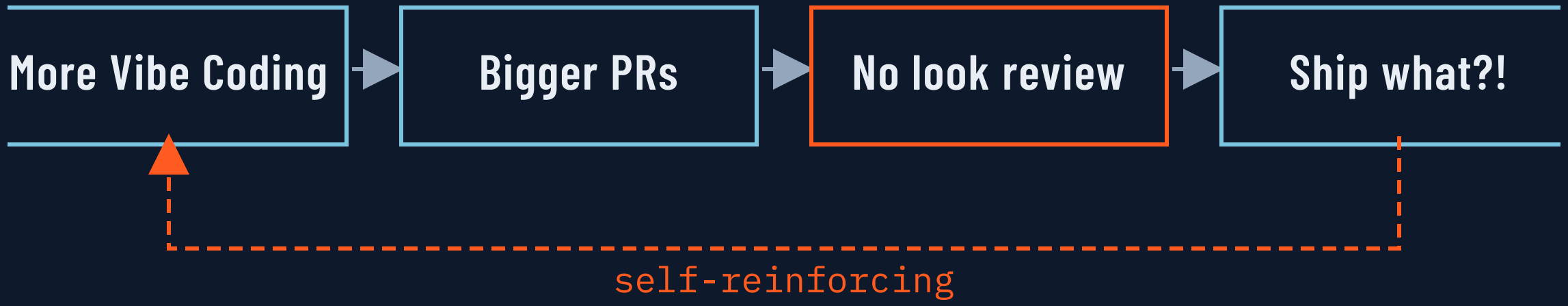


THE SLOP

I got it working on my machine!

Can’t you just make it run in production?

The loop feeds itself



Personal Example

499K

lines in one PR.

- Two days of Vibe coding over the weekend
- PR approved in under 20 minutes.
  - That’s 415 lines of code per second.

Maintenance scales with volume

IF WE WRITE

1,000 lines

and it costs $X to manage



THEN

10,000 lines

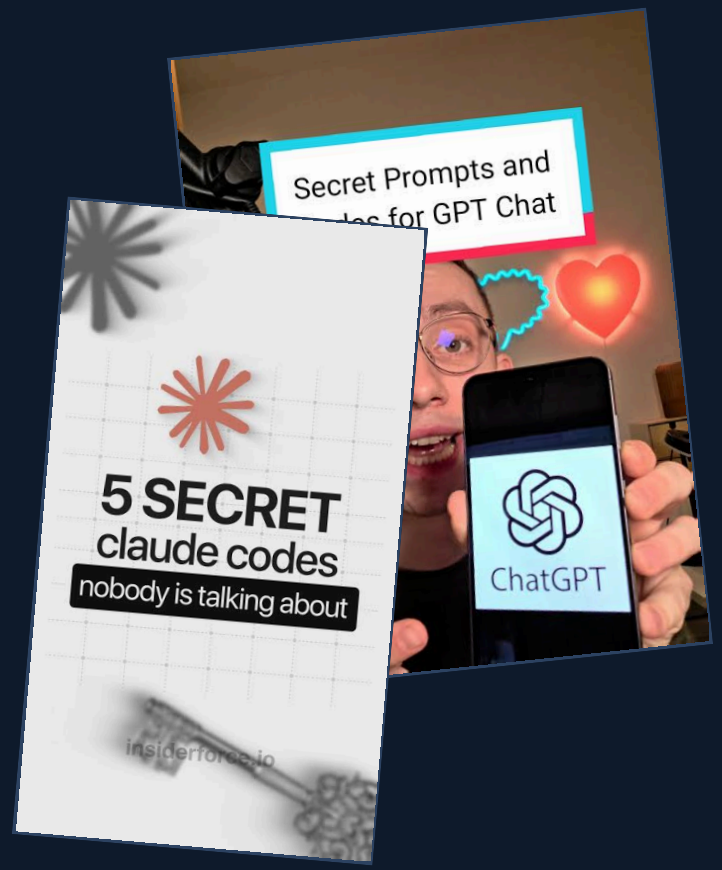
costs at least 10X



EXPLICIT IS BETTER THAN NOTHING

What we are *not* talking about

No magic words. No secret codes.





Specifications Ideas.

- **Literate programming** — Knuth, 1984: the explanation of intent is the primary artifact, code is derived from it
- **ADR via RFC** — MUST / SHOULD / MAY, a shared vocabulary to remove ambiguity
- **ASD-STE100** — Simplified Technical English, a controlled vocabulary and grammar.
- **Spec-driven tools** — Speckit and others apply the same idea to agents



LIMITS OF EXPLICIT

Limited value

- Explicit instructions help, but only so much
- "Magic words" and hidden-trick prompts... snake oil.
- The effect is tied to the model, and is brittle.
- Explicit effectiveness is rarely measured.

Prompt Evaluation

Dan Luu: agents implement **Zstd in Rust**, graded against hidden tests — 26 prompt conditions, ~160 runs each.

- **Techniques tested** — TDD, property-based testing, fuzzing, mutation testing, differential testing, formal methods (TLA+, Lean 4, Verus, SMT solvers), and more
- **None of it moved the needle** — naming a technique didn't change outcomes
- **Tests were poorly written, and copied mistakes**

Dan Luu, danluu.com/agentic-testing



DETERMINISTIC FEEDBACK

Power of Ten

- Holzmann, JPL, 2006
- Ten rules for safety-critical C
- Written so tools can check them, not reviewers' judgment

ntrs.nasa.gov

AUTOMATE AS MUCH OF CODE REVIEW AS POSSIBLE, SO A HUMAN DOESN'T HAVE TO.

When the AI touches a check, flag it

- Run the harness in a container
- Checks mounted read-only, so the agent cannot modify them
- Your machine is the first test environment.

Verification lives *outside the generation loop.*

5 BEYOND UNIT TESTING

QUIS CUSTODES IPSOS CUSTODES?

(Who watches the watchman?)

Hint: Not AI.

What do passing tests prove?

If AI writes the code *and* all the tests, they prove only that the code agrees with the AI's own reading of the task.

- Dan Luu did find that independent test generation was useful.

Layers of functional testing

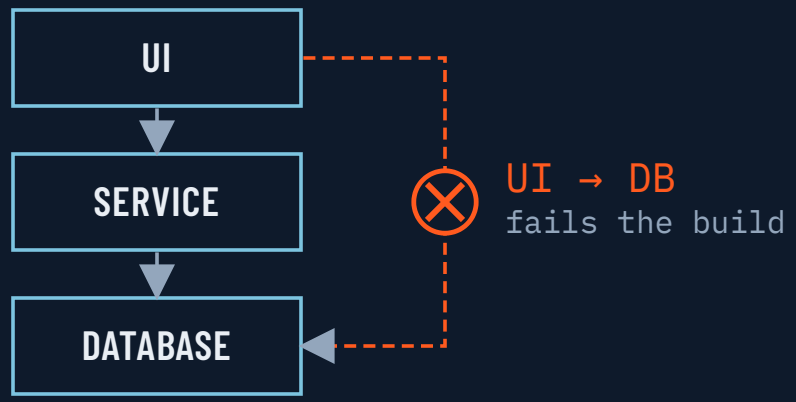
LAYER	CHECKS
Unit	Just the business logic
Integration	Components working together
End-to-end / acceptance	Behavior against the spec

6 ARCHITECTURE FITNESS

Fitness function

- An automated check on a structural property
- From *Building Evolutionary Architectures* (Ford, Parsons, Kua)
- Functional tests: does it do the right thing?
- Fitness: is it still built the right way?

A forbidden dependency



- Dependency direction
- Layers and module boundaries
- No cycles
- Coupling and size limits
- Fails the build in CI

Tools

ECOSYSTEM	TOOL
Java	ArchUnit
JavaScript / TypeScript	dependency-cruiser
Python	import-linter
.NET	NetArchTest

7

COMPLEXITY LIMITS

File and function level

Complexity caps

- Cyclomatic: independent paths through a function (McCabe, 1976)
- Cognitive: how hard it is to read (SonarSource)
- Max nesting depth and parameter counts too
- CRAP score: complexity combined with test coverage, in one gate (Savoia and Evans, 2007)

testing.googleblog.com/2011/02/this-code-is-crap.html

Size and duplication

- Function length: Power of Ten rule 4 sets about 60 lines (Holzmann)
  - Personally: Always fit in a screen.
- Clean code: "Functions should do one thing. They should do it well. They should do it only."
- File length

AUTOMATED VALIDATIONS HOLD DOWN COST PER LINE.



BEYOND FUNCTIONAL TESTS

Mutation testing

- Make small changes to the code
- Tests still pass? The mutant survived. The tests are too weak.
- Google runs it on code changes in review (Petrovic and Ivankovic, 2018)
- PIT (Java), Stryker (JS, TS, C#), mutmut (Python)

pitest.org/quickstart/basic_concepts

Property-based testing

- State a rule that must hold for all inputs
- The tool generates the cases
- Failures shrink to a minimal example

QuickCheck, Claessen and Hughes, 2000 · How Hypothesis works

Fuzzing

- Randomly generate inputs

24–33% of ~90 UNIX utilities crashed or hung on random input

Miller, Fredriksen, and So, 1989

Structure-aware fuzzing

- Builds inputs that follow the target's format
- Pure random bytes mostly hit the rejection paths

Differential and metamorphic

- Differential: same inputs, two implementations, investigate disagreements (McKeeman, 1998)
  - The second must be independent, or both carry the bug
- Metamorphic: no known output? Transform the input, check how the output should change (Segura et al., 2016)

Choose by what you can state

YOU CAN STATE	USE
A rule	Property-based
A reference	Differential
A relationship	Metamorphic
Survival	Fuzz

Closing Thoughts

- Validations lower the cognitive load of review.
- Deterministic tests validate function.
- Advanced tests find faults through patterns.

THANKS.

linkedin.com/in/stephenshary